



CYBERCLASS

Capacitando el presente para el futuro

Participante: Darien Ruben Suazo De Jesus

Técnico: Ciberseguridad

Módulo: CRIPTOGRAFÍA Y CIFRADO DE DATOS

Facilitador: Kevin Feliz Henríquez

Centro: Cyberclass

Práctica: Fundamentos de Criptografía

Descripción: En esta documentación verán el paso a paso de como hacer un programa que cifre y descifre el cifrado cesar

¿Qué es el Cifrado de César?

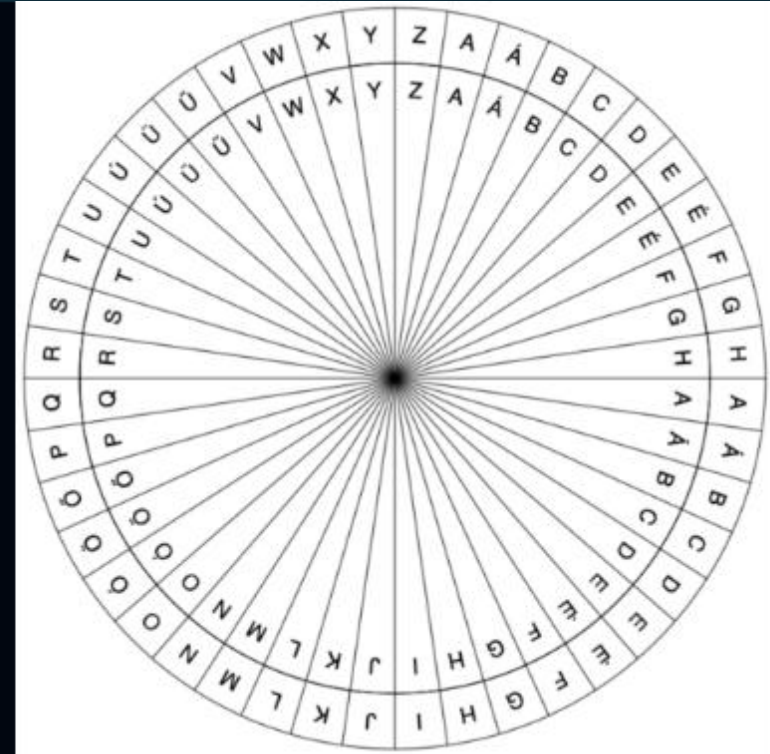
Sustitución Monoalfabética

El Cifrado de César consiste en desplazar secuencialmente cada carácter del alfabeto por un número fijo de posiciones (**clave**).

Para un alfabeto estándar de 26 letras:

$$C = (P + K) \bmod 26$$

Donde **C** es el carácter cifrado, **P** el original, y **K** la clave de desplazamiento.



El Menú Principal e Interfaz

app.py > mostrar_menu()

```
def mostrar_menu():  
    print("\n==== CIFRADO DE CESAR =====")  
    print("1. Cifrar un mensaje")  
    print("2. Descifrar un mensaje")  
    print("3. Ver historial de cifrados")  
    print("4. Salir")
```

Terminal - Python Shell

C:\Users\Usuario\Desktop\cifrado_cesar> py app.py

```
==== CIFRADO DE CESAR =====  
1. Cifrar un mensaje  
2. Descifrar un mensaje  
3. Ver historial de cifrados y descifrados  
4. Salir
```

Lógica Interna de Cifrado

```
● ● ● app.py > cifrar_cesar()
```

```
def cifrar_cesar(texto, clave):  
    resultado = ""  
    for char in texto:  
        if char.isalpha():  
            desplazamiento = clave % 26  
            if char.islower():  
                resultado += chr((ord(char) - ord('a') +  
                                desplazamiento) % 26 + ord('a'))  
            elif char.isupper():  
                resultado += chr((ord(char) - ord('A') +  
                                desplazamiento) % 26 + ord('A'))  
    return resultado
```

Ajuste Dinámico por ASCII

- **Control de Rango:** El operador `% 26` previene desbordamientos si el usuario introduce una clave mayor a 26.
- **Fidelidad de Capitalización:** Identifica mayúsculas y minúsculas por separado usando la referencia de caracteres ASCII `ord('a')` y `ord('A')`.

Ejecución de Cifrado Activo

Caso de Prueba: "golazo"

Análisis matemático del desplazamiento secuencial:

- **Mensaje:** golazo
- **Clave de Entrada:** 23
- **Cálculo:**
Cada carácter rota 23 posiciones adelante en el alfabeto de 26 posiciones.
- **Resultado:** dlixwl

Terminal - Prueba de Cifrado

```
Seleccione una opción: 1
Ingrese el mensaje a cifrar: golazo
Ingrese la clave de cifrado (número entero): 23
Mensaje cifrado: dlixwl
```

Proceso de Descifrado Reversible

```
app.py > descifrar_cesar()
```

```
def descifrar_cesar(texto, clave):  
    return cifrar_cesar(texto, -clave)
```

Principio de Simetría

En lugar de reescribir toda la lógica algebraica de caracteres, el software aplica un diseño inteligente:

$$P = (C - K) \text{ mod } 26$$

Al pasar la **clave negativa** (-clave) a la función de cifrado principal, el desplazamiento rota automáticamente hacia la izquierda de forma matemática.

↶ Reversión del Mensaje en Acción

Desplazamiento Negativo

Se ingresa el texto encriptado para verificar que el algoritmo es reversible:

- **Mensaje Cifrado:** `dlixwl`
- **Clave de Descifrado:** `23` (Equivale a rotar 23 posiciones atrás).
- **Salida Recuperada:** `golazo`

Terminal - Prueba de Descifrado

```
Seleccione una opción: 2
Ingrese el mensaje a descifrar: dlixwl
Ingrese la clave de descifrado (número entero): 23
Mensaje descifrado: golazo
```

Gestión y Almacenamiento de Historial



Estructura LIFO

Inicialización de un arreglo vacío dinámico en la memoria temporal para almacenar las transacciones.

```
historial = []
```



Inserción Dinámica

Uso del método nativo de Python para registrar una tupla que empareja el texto y la clave.

```
historial.append((msg, clave))
```



Renderización

Función de ciclo for encargada de iterar el historial y formatear la consola para la vista del usuario final.

Salida de Historial

 app.py > mostrar_historial()

```
def mostrar_historial():  
    print("\n==== HISTORIAL DE CIFRADOS =====")  
    for mensaje, clave in historial:  
        print(f"Mensaje: {mensaje}, Clave: {clave}")
```

 Terminal - Historial de Operaciones

Seleccione una opción: 3

==== HISTORIAL DE CIFRADOS Y DESCIFRADOS =====

Mensaje: dlixwl, Clave: 23

Mensaje: golazo, Clave: 23

🔗 Ciclo de Control de Flujo Continuo

Experiencia de Usuario Fluida

Para evitar tener que reiniciar la consola después de cada proceso, la función `principal()` ejecuta un ciclo anidado.

- Pregunta recursivamente: "**¿Desea cifrar otro mensaje?**"
- Captura las entradas 's' / 'n' de forma segura.
- Regresa al menú de navegación principal de forma automática y limpia.

Terminal - Ejecución Múltiple

```
Seleccione una opción: 1
Ingrese el mensaje a cifrar: beyblade
Ingrese la clave: 10
Mensaje cifrado: loilvkno
¿Desea cifrar otro mensaje? (s/n): s
Ingrese el mensaje a cifrar: vuela
Ingrese la clave: 5
Mensaje cifrado: azjqf
¿Desea cifrar otro mensaje? (s/n): n
Regresando al menú principal...
```

Resumen de lo aprendido

- ▶ El funcionamiento el codigo cesar
- ▶ Usar el codigo cesar para cifrar
- ▶ Implementar el codigo cesar como parte de un programa